

Healthcare Data Analysis & ETL Documentation

SQL Data Cleaning • Analytics Queries • Python Exploratory Data Analysis

Technical Summary: This document outlines the end-to-end data pipeline applied to the hospital dataset ('healthcare_dataset'). It includes comprehensive SQL transformation and cleaning scripts, strategic SQL insight extraction queries, and Python profiling for demographic, billing, and clinical pattern identification.

1. SQL DATA CLEANING & TRANSFORMATIONS

Database Engine: MySQL / PostgreSQL

The raw data underwent text standardization, duration calculation, negative value adjustments, deduplication using window functions, and primary key schema enhancements.

```
-- Database Selection & Text Formatting
USE healthcare_db;

-- Format Name column to Proper Title Case
UPDATE healthcare_dataset
SET Name = CONCAT(
    UPPER(SUBSTRING(Name, 1, 1)),
    LOWER(SUBSTRING(Name, 2, LOCATE(' ', Name) - 1)),
    ' ',
    UPPER(SUBSTRING(Name, LOCATE(' ', Name) + 1, 1)),
    LOWER(SUBSTRING(Name, LOCATE(' ', Name) + 2))
)
WHERE Name LIKE '% %';

-- Standardize Billing Amount Precision
UPDATE healthcare_dataset
SET `billing amount` = ROUND(`billing amount`, 2);

-- Calculate Length of Stay (Stay_Duration_Days)
ALTER TABLE healthcare_dataset ADD COLUMN Stay_Duration_Days INT;
UPDATE healthcare_dataset
SET Stay_Duration_Days = DATEDIFF(
    STR_TO_DATE(`Discharge Date`, '%Y-%m-%d'),
    STR_TO_DATE(`Date of Admission`, '%Y-%m-%d')
);

-- Correct Anomaly: Convert Negative Billing Amounts to Positive
UPDATE healthcare_dataset
SET `Billing Amount` = ABS(`Billing Amount`)
WHERE `Billing Amount` < 0;

-- Deduplication: Remove Exact Overlapping Patient Admissions
CREATE TABLE healthcare_clean AS
SELECT * FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY Name, `Date of Admission`, `Medical Condition`
            ORDER BY `Discharge Date` DESC
        ) AS row_num
    FROM healthcare_dataset
) AS temp
WHERE row_num = 1;

ALTER TABLE healthcare_clean DROP COLUMN row_num;
DROP TABLE healthcare_dataset;
RENAME TABLE healthcare_clean TO healthcare_dataset;
```

2. STRATEGIC SQL ANALYTICS INSIGHTS

Analytical Queries

Clinical & Financial Queries: Targeted SQL statements designed to measure treatment efficacy, emergency blood profile dependencies, abnormal risk factors, and revenue drivers across patient demographics.

```
-- Insight 1: Highest Normal Test Result Rate by Condition & Medication
SELECT
    `Medical Condition`,
    `Medication`,
    COUNT(*) AS Total_Patients,
    ROUND(AVG(CASE WHEN `Test Results` = 'Normal' THEN 100.0 ELSE 0.0 END), 1) AS
Normal_Result_Rate_Pct
FROM hospital_data
GROUP BY `Medical Condition`, `Medication`
ORDER BY Normal_Result_Rate_Pct DESC
LIMIT 5;

-- Insight 2: Emergency Patient Breakdown by Blood Type
SELECT
    `Blood Type`,
    COUNT(*) AS Emergency_Patients_Count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS Share_Pct
FROM hospital_data
WHERE `Admission Type` = 'Emergency'
GROUP BY `Blood Type`
ORDER BY Emergency_Patients_Count DESC;

-- Insight 3: Abnormal Test Risk & Average Stay for Emergency Admissions
SELECT
    `Medical Condition`,
    COUNT(*) AS Emergency_Patients,
    ROUND(AVG(CASE WHEN `Test Results` = 'Abnormal' THEN 100.0 ELSE 0.0 END), 1) AS
Abnormal_Risk_Pct,
    ROUND(AVG(Stay_Duration_Days), 1) AS Avg_Stay_Days
FROM hospital_data
WHERE `Admission Type` = 'Emergency'
GROUP BY `Medical Condition`
ORDER BY Abnormal_Risk_Pct DESC;

-- Insight 4: Conditions Driving Extended Hospital Stays (>20 Days)
SELECT
    `Medical Condition`,
    COUNT(*) AS Long_Stay_Patients,
    ROUND(AVG(Stay_Duration_Days), 1) AS Avg_Stay_Days
FROM hospital_data
WHERE Stay_Duration_Days > 20
GROUP BY `Medical Condition`
ORDER BY Long_Stay_Patients DESC;

-- Insight 5: Revenue & Stay Metrics by Admission Pathway
SELECT
    `Admission Type`,
    ROUND(AVG(Stay_Duration_Days), 1) AS Avg_Stay_Days,
    ROUND(AVG(`Billing Amount` * 1.0 / Stay_Duration_Days), 0) AS Daily_Revenue_USD
FROM hospital_data
GROUP BY `Admission Type`
ORDER BY Daily_Revenue_USD DESC;

-- Insight 6: Senior (>60) vs. Non-Senior Demographic Comparison
SELECT
    CASE WHEN Age > 60 THEN 'Senior (>60)' ELSE 'Non-Senior (<=60)' END AS Demographic_Group,
    COUNT(*) AS Total_Patients
```

Data Import & Type Conversion

Parsing raw TSV exports, casting dates into Datetime objects, and verifying column integrity.

Summary Metrics Generation

Calculating core dataset KPIs including billing totals, mean/median stays, and demographics.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load Cleaned Dataset & Date Preprocessing
df = pd.read_csv("healthcare_cleaned.csv", sep="\t")
df["Date of Admission"] = pd.to_datetime(df["Date of Admission"], format="mixed")
df["Discharge Date"] = pd.to_datetime(df["Discharge Date"], format="mixed")

# 2. Overall KPI & Descriptive Statistics
total_records = len(df)
total_billing = df["Billing Amount"].sum()
average_billing = df["Billing Amount"].mean()
average_stay = df["Stay Duration Days"].mean()

print(f"Total Records: {total_records} | Total Billing: ${total_billing:,.2f}")
print(f"Avg Billing: ${average_billing:,.2f} | Avg Length of Stay: {average_stay:.1f} days")

# 3. Demographic Binned Grouping (Age Distribution)
df["Age Group"] = pd.cut(
    df["Age"],
    bins=[0, 18, 30, 45, 60, 75, 100],
    labels=["0-18", "19-30", "31-45", "46-60", "61-75", "76-100"]
)

# 4. Medical Condition Financial Aggregation
billing_by_condition = (
    df.groupby("Medical Condition")["Billing Amount"]
    .agg(["count", "sum", "mean", "median"])
    .sort_values("mean", ascending=False)
)

# 5. Temporal Trend Analysis (Admissions by Year & Month)
df["Admission Year"] = df["Date of Admission"].dt.year
df["Admission Month"] = df["Date of Admission"].dt.month
monthly_admissions = df["Admission Month"].value_counts().sort_index()

plt.figure(figsize=(9, 4))
sns.lineplot(x=monthly_admissions.index, y=monthly_admissions.values, marker="o", color="#2563eb")
plt.title("Monthly Patient Admission Trends")
plt.xlabel("Month")
plt.ylabel("Admissions")
plt.tight_layout()
plt.show()
```

High-Cost Threshold Analysis: Isolating top 10% outlier billing cases (90th percentile) to identify conditions driving disproportionate medical expenditure.

```
# 6. High-Cost Outlier Identification (90th Percentile)
high_cost_threshold = df["Billing Amount"].quantile(0.90)
df["High Cost"] = np.where(
    df["Billing Amount"] >= high_cost_threshold,
    "High Cost",
    "Regular"
)

high_cost_by_condition = (
    df[df["High Cost"] == "High Cost"]["Medical Condition"]
    .value_counts()
)

# 7. Comprehensive Condition Summary Aggregation
condition_summary = (
    df.groupby("Medical Condition")
    .agg(
        Records=("Medical Condition", "size"),
        Average_Billing=("Billing Amount", "mean"),
        Total_Billing=("Billing Amount", "sum"),
        Average_Stay=("Stay_Duration_Days", "mean"),
        Average_Age=("Age", "mean")
    )
    .sort_values("Records", ascending=False)
)

# 8. Test Result Distribution (%) per Condition
test_result_pct = pd.crosstab(
    df["Medical Condition"],
    df["Test Results"],
    normalize="index"
) * 100

# 9. Correlation Matrix across Numeric Indicators
numeric_cols = ["Age", "Billing Amount", "Stay_Duration_Days", "Daily_Billing_Amount"]
correlation_matrix = df[numeric_cols].corr()
print("Correlation Matrix:\n", correlation_matrix.round(3))
```

KEY FINDINGS & EXECUTIVE SUMMARY

- **Billing Anomalies Corrected:** Negative billing amounts were converted using absolute transformation, and name fields were cleaned to proper Title Case.
- **Length of Stay (LOS) Impact:** Average length of stay holds steady around 15.5 days across major conditions, with low linear correlation to total billing.
- **High-Cost Concentration:** Obesity and Hypertension represent the top billing averages, requiring targeted resource and bed-utilization optimization.